

Projektovanje složenih digitalnih sistema

Predavanje VI

Lecture Content

- Design Optimization
  - Introduction
  - Code Transformations
    - Bit-Level Transformations
    - Instruction-Level Transformations
    - Loop-Level Transformations
    - Data-Oriented Transformations



Introduction

Introduction

- In order to achieve desired performance levels, in terms of execution speed, design size, power consumption, etc., designers can use a wide range of optimization techniques

- Using optimization techniques original algorithm is transformed into an equivalent form that, when implemented in hardware, will result in more efficient implementation

- Optimization techniques can be divided into two large groups:
  - **Code Transformations**
  - **Mapping and Execution Transformations**



Code Transformations

Code Transformations

- These optimizations are applied directly to the pseudo-code of the algorithm

- These optimizations expose existing concurrency and data locality, present in the algorithm, which than can se used, on different levels to achieve more efficient hardware implementation

- Optimizations that fall into this group are:
  - Bit-Level Transformations
  - Instruction-Level Transformations
  - Loop-Level Transformations
  - Data-Oriented Transformations



Bit-Level Transformations

Bit-Level Transformations

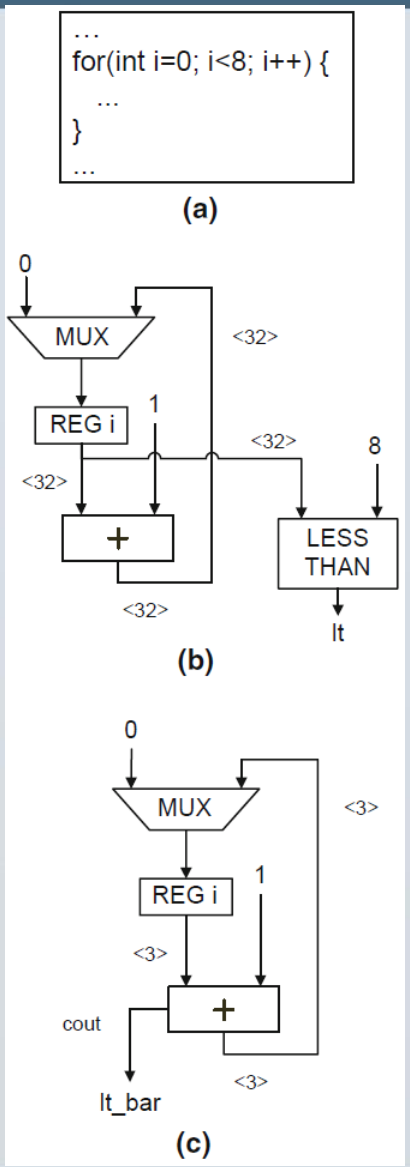
- Bit-level transformations aim at exposing to the implementation the amount of bit-level resources strictly needed to carry out the arithmetic or logical operations at hand
- Using these optimization architectures can leverage the bit-level information through customization and specialization to deliver implementations with substantially fewer hardware resources than direct and naive implementations
- In many cases, these optimized implementations may exhibit faster clock rates as the critical path of the generated data-paths is shortened
- Best known bit-level transformations are:
  - Bit-Width Narrowing,
  - Bit-Level Optimizations, and
  - Conversion between floating-point and fixed-point data formats

Bit-Width Narrowing

- In many algorithms, the declared precision and range of the numeric data types used are overly-defined
- This over-definition occurs when the bit-widths of the data types are much larger than the bit-widths required to store the data for the observed values during the execution of the computations
- By analyzing the actual value ranges each variable in the algorithm can take during the execution of the algorithm, optimal (minimal) bit-widths can be calculated
- There are two approaches when determining the optimum bit-width representations of variables:
  - Static bit-width analysis
  - Run-time profiling

Bit-Width Narrowing Illustrative Example

- An illustrative example of this over-definition occurs by direct implementation in hardware of the arithmetic and logic operations used in loop control variables as depicted in Figure *a)*
- In this example, the native 32-bit two's complement integer representation for the loop's control variable *i* allows for a range of integer values of $[-2^{31}, 2^{31}-1]$, whereas in reality only 3 bits are required to span the range of values [0,7]
- A naive direct implementation of the controller for the execution of the loop would use the default 32-bit adder in the hardware solution depicted in Figure *b)* where the (i<=7) comparison relies on a comparator
- Given the information about the range of values assumed by the variable *i*, designer could generate an hardware solution using a single 3-bit unsigned adder operator instead.
- Figure *c)* depicts an even more aggressive hardware implementation variant where the carry-out signal (*cout*) of the 3-bit adder is used to indicate if the value of the variable *i* is less than 8 resulting in an extremely compact hardware design solution

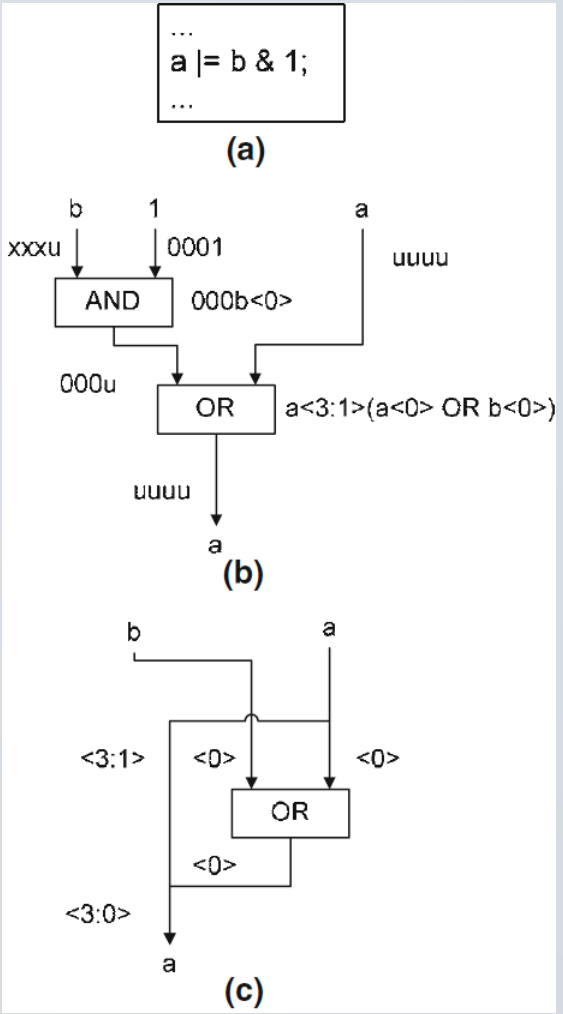


Bit-Level Optimizations

- Bit-level optimizations refer to optimizations of logic functions, possibly performed at gate-level
- Typically, they should be performed by the RTL compiler
- These optimizations are based on applying traditional boolean minimization techniques and result in simplified boolean expressions that are then mapped into hardware using fewer logic gates and lower execution delay
- Bit-level optimizations may also use the bit-value results, gather by bit-width related analyses described earlier, to simplify, or even eliminate, arithmetic and logic operations

Bit-Level Optimizations Illustrative Example

- Figure on the right illustrates an example of these simplifications considering for simplicity 4-bit data representations
- The initial 4-bit && and || operations in Figure *a)* can be directly translated to 4-bit logic AND and OR gates as depicted in Figure *b)*
- Using the information about the bit-values of the integer constant 1, the compiler converts the 4-bit logic AND and OR gates to a single wire and a 1-bit logic OR gate, respectively as depicted in Figure *c)*
- Bit optimizations can even allow a compiler to eliminate the hardware that, in other case, would be associated with a given statement, as is the case of **if**((*ans*&0x8000)==0x8000) which can be implemented in hardware using a simple wire connected to the 15th bit of *ans*
- Similarly, the evaluation of predicates of the form (a<0) and (a>=0) can be performed by inspecting the logic value of the most significant bit of a assuming the sign of the representation identified by that bit



Conversion between Floating-point and Fixed-point Data Formats I

- Floating-point data representation is a commonly used format to represent real numbers in almost all computing domains. In the IEEE 754 single precision floating-point standard each floating-point value is represented as:

$$(+/-)1.f \times 2^{(exp-127)}$$

- where *f* represents the fraction (mantissa) using 23 bits and *exp* represents the 8-bit exponent

- The costs associated to the arithmetic operations in floating-point formats, however, are much higher than using an integer representation in terms of logic gates and timing

Conversion between Floating-point and Fixed-point Data Formats II

- An alternative representation, commonly used in digital signal/image processing algorithm implementations, are the fixed-point data types
- The fixed-point data types allow the arithmetic operations to be done by integer operations and scaling factors implemented by shifting the operands and/or the result of the operations
- In some cases the trade-off between precision and efficiency of computations in fixed-point format is acceptable given the substantial reduction of the costs of the arithmetic operations
- Given that most algorithms have been developed on computers natively supporting floating-point data types, porting floating-point based codes to use fixed-point data types requires an error-prone, tedious, and time-consuming conversion process



Instruction-Level Transformations

Instruction-Level Transformations I

- At a coarser level of granularity in code transformations, many instruction-level transformations can be used
- These transformations increase performance and/or simplify or reduce the hardware resources allocated for a given computation using a combination of algebraic simplification or circuit specialization
- As instructions are translated into specific hardware operators such as adders or multipliers, the simplification or even elimination of operations (in the absence of resource sharing) directly corresponds to the elimination of hardware resources in the circuit that implements the desired computation

Instruction-Level Transformations II

- Simple algebraic transformations can often have a positive impact on the generated hardware, such as:
  - Common Subexpression Elimination,
  - Constant Folding and Constant Propagation
- Other algebraic transformations, as is the example of replacing a square operation with a multiplication (a^2 replaced by $a \times a$), are useful when an architecture does not natively support the original operation
- We focus here also on the following three key transformations enabled or emphasized by the flexibility of hardware architectures:
  - Operator Strength Reduction
  - Tree-Height Reduction
  - Code Motion

Common Subexpression Elimination

- **Common Subexpression Elimination** (CSE) searches for instances of identical expressions (i.e., they all evaluate to the same value), and analyzes whether it is worthwhile replacing them with a single variable holding the computed value

- For example, look at the following code

```
a = b * c + g;
d = b * c * e;
```

- It may be worth transforming the code to

```
tmp = b * c;
a = tmp + g;
d = tmp * e;
```

- If the time cost (savings) of storing and retrieving values of new variable *tmp* outweighs the cost of calculating operation b*c an extra time

Constant Folding and Constant Propagation I

- **Constant folding** is the process of recognizing and evaluating constant expressions at compile time rather than computing them at runtime
- **Constant propagation** is the process of substituting the values of known constants in expressions at compile time
- Constant folding and propagation are typically used together to achieve many simplifications and reductions, by interleaving them iteratively until no more changes occur
- Consider the following pseudo-code

```
int a = 30;
int b = 9 - (a / 5);
int c; c = b * 4;
if (c > 10) {c = c - 10;}
return c * (60 / a);
```

Constant Folding and Constant Propagation II

- Applying constant propagation once, followed by constant folding, yields

```
int a = 30;
int b = 3;
int c;
c = b * 4;
if (c > 10) {c = c - 10;}
return c * 2;
```

- Repeating both steps twice results in:

```
int a = 30;
int b = 3;
int c;
c = 12;
if (true) { c = 2; }
return c * 2;
```

Constant Folding and Constant Propagation III

- As *a* and *b* have been simplified to constants and their values substituted everywhere they occurred, we can now apply **Dead Code Elimination** to discard them, reducing the code further:

```
int c;
if (true) { c = 2; }
return c * 2;
```

- Since the *if* statement will always evaluate to true, *c* itself can be eliminated, shrinking the code even further:

```
return 4;
```

Operator Strength Reduction I

- **Operator Strength Reduction** (OSR) replaces a specific operation with a sequence of less expensive operations

- Strength reduction can achieve resource savings and reduce the delay of the operation, and is therefore well suited for hardware implementations given their ability to implement specific operations by direct manipulation of wires and the ability to customize and/or combine operators

- A first class of strength reduction is usually applied to induction variables, as is the example of replacing

$$j=i*2$$

- with

$$j=j+2$$

- for loops with *i* control variable and unit step

Operator Strength Reduction II

- A second class of strength reduction transformations contains simple bit manipulation operations that can be trivially implemented by changing the meaning of the wires that carry the values of the variables, thereby eliminating altogether the “instructions” or operators

- As an illustrative example, the operation

$$2 \times i$$

- can be replaced by the operation

$$i << 1$$

- which in hardware can be implemented using simple wire connections

Operator Strength Reduction III

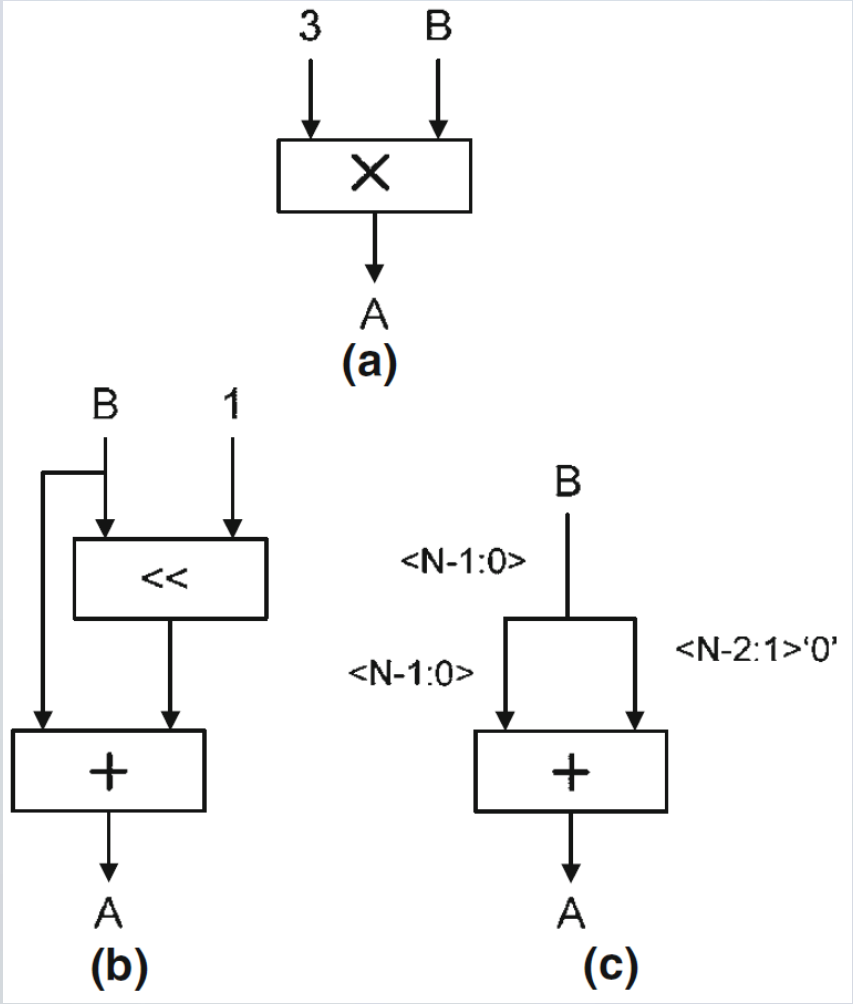
- For example, the operation

$$3 \times i$$

- can be more efficiently implemented as

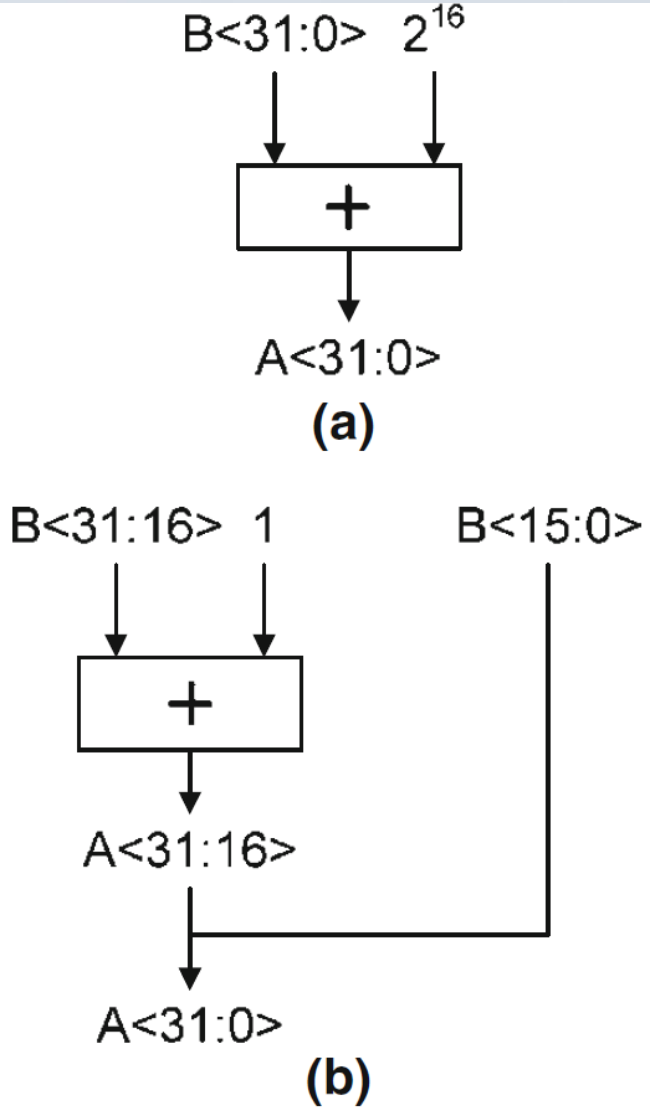
$$i+(i<<1)$$

- Figure on the right depicts an illustrative implementation of the latter operation strength reduction example



Operator Strength Reduction IV

- A third class of strength reduction specializes an operation based on the value of its operands
- This is particularly relevant in the context of arithmetic operations given the computational weight and amount of resources needed to implement these operators
- For example, when a constant of the form 2^N is added to an operand, an increment and a bit-level concatenation can be used instead of a full adder.
- Figure on the right depicts one particular example, where a 32-bit operand is added to the constant 2^{16}, thus only requiring a 16-bit increment unit rather than a 32-bit adder



Operator Strength Reduction V

- Generically, integer divisions and multiplications by compile-time constants can be transformed into sequences of shifts, additions, and subtractions
- Trivial cases occur when there is a multiplication or a division of an integer operand by a power-of-two constant
- For these cases, a multiplication is accomplished by a simple shift of the operand, which, in hardware implementations, requires only wire interconnections

Height Reduction I

- Height reduction is an instrumental transformation when mapping arithmetic expressions to hardware
- It rearranges the way operations are performed preserving the functionality of the original computations
- By exploiting commutative, associative, and distributive properties of arithmetic operations, a compiler using height reduction can reduce the number of operations in an expression, or/and the critical path of the resulting hardware implementation
- A special case of height reduction is **Tree-Height Reduction** (THR) which is applied to operations organized as a tree
- By reducing the height of an expression tree, the technique exposes operator concurrency and consequently reduces the latency of the hardware circuit that implements the expression

Height Reduction II

- Simple uses of tree-height reduction, such as in

$$t=a+b+c+d$$

- only require the application of arithmetic associativity to derive

$$t=(a+b)+(c+d)$$

- In other arithmetic expressions, distributivity is a key transformation that exposes a wide variety of hardware implementation trade-offs

- For this reason we focus on distributivity in the remainder of this section

Height Reduction III

- Often, the distributive property leads to implementations with more operations and without lower latency, but in some cases distributivity can expose subexpressions common to other expressions in the algorithm
- For instance, the application of distributivity to the instruction sequence:

  t1=a*(b+c+d); t2=a*b+a*d;
- leads to the transformed instruction

  t1=a*b+a*c+a*d;
- which exposes the common subexpression *a*b+a*d* resulting in the code sequence

  t2=a*b+a*d; t1=t2+a*c;
- Without any resource sharing, direct hardware implementations of both sequences have the same latency of 2*Lat(+)+Lat(*)
- The transformed sequence, however, requires only two additions and three multiplications whereas the original code would require three additions and three multiplications

Height Reduction IV

- Distributivity can also be used to reduce the latency of a hardware implementation, as illustrated by the statement

t1=a*(b*c*d+e);

- The transformed code after applying distributivity results in

t1=(a*b)*(c*d)+a*e;

- which requires one additional multiplication
- Its hardware implementation exhibits a latency of

$2\times Lat(*)+Lat(+)$ clock cycles

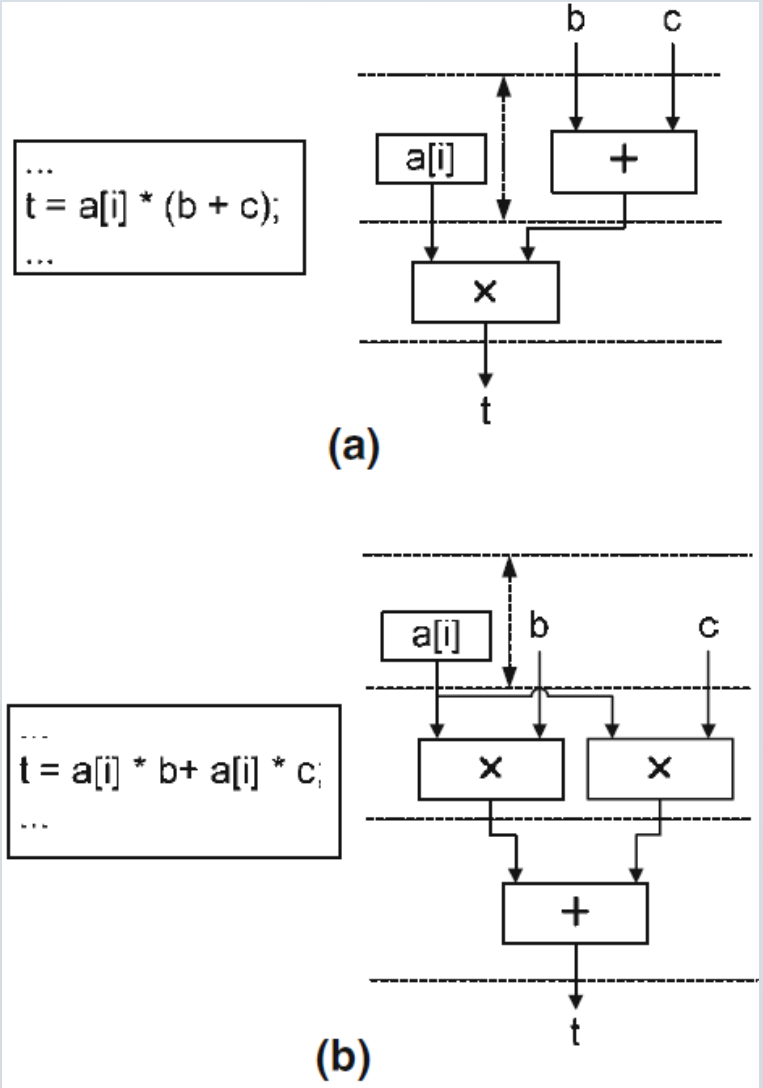
- rather than

$3\times Lat(*)+Lat(+)$ clock cycles

- for the hardware implementation of the original statement

Height Reduction V

- There are cases, however, where distributivity leads to performance degradation and an increase in the number of operations (i.e., and consequently hardware resources) as illustrated in the example in Figure on the right
- Without distributivity the memory read, *a[i]*, can be performed in parallel with the addition *(b+c)*, which can result in a latency of two clock cycles, when considering one cycle to read data from memory
- The implementation using distributivity does not allow this parallelism and the memory read for *a[i]* must be done before the additions
- This results in a latency of three clock cycles



Height Reduction VI

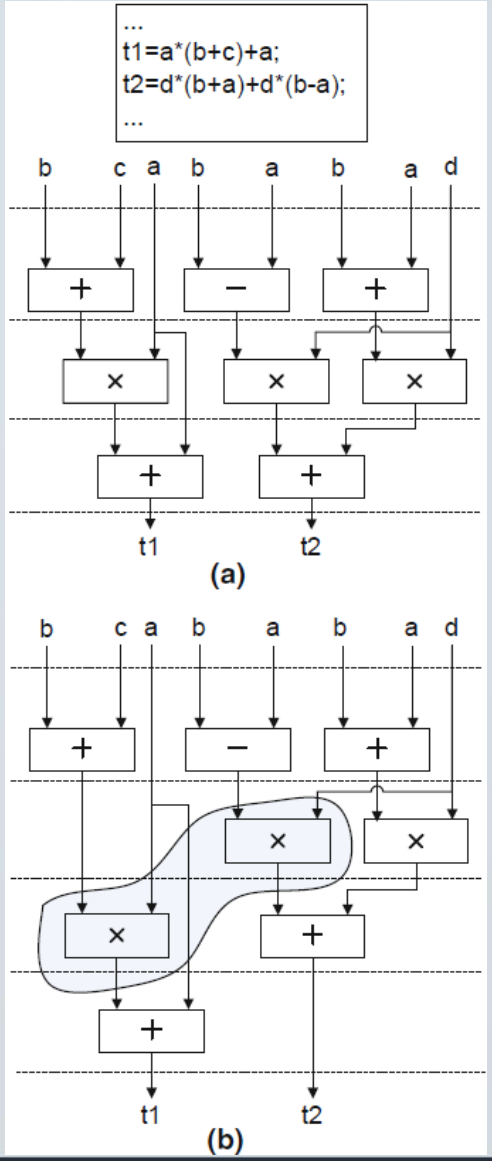
- Although distributivity does increase the number of operators in a given expression, its application may break dependences leading to shorter execution schedules when the number of hardware resources is limited

- Figure on the right illustrates this scenario for the original code in Figure *a),* without constraints on the number of multipliers used in parallel and for hardware implementations constrained to two hardware multipliers, shown in Figure *b)*

- The hardware implementation of the original code would have to implement the multiplication in statement *t1=...* in a next cycle due to the dependence on the addition in the same statement

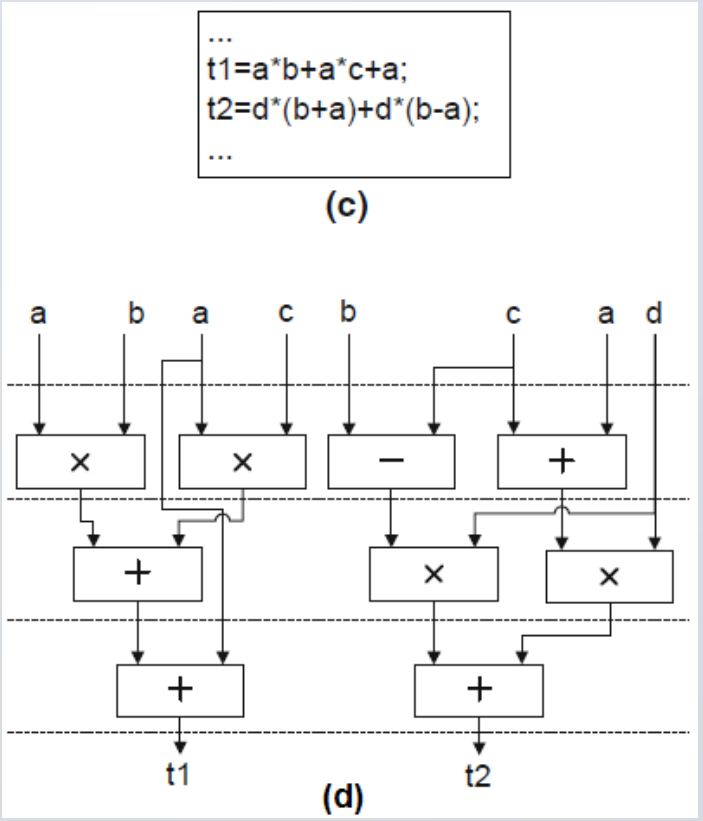
- This would imply that in a second cycle one would have three multiplications, one corresponding to *t1= ...* and two multiplications corresponding to the statement *t2=....*

- Given the implementation constraint of two multipliers, these three multiplications would have to be carried out in two additional steps, leading to a schedule with four overall execution steps, as depicted in Figure *b)*



Height Reduction VII

- Let us apply distributivity to the statement t1=… from the original code shown in Figure a) on the previous slide
- Resulting code is shown in Figure *c)*
- The multiplications corresponding to statement *t1=...* now can be performed during a first execution step using the two available multipliers
- The multiplications corresponding to statement *t2=...* can be carried out in a second execution step concurrently with the additions for statement *t1=...*
- Resulting schedule is shown in Figure *d),* with an overall execution latency of only two execution steps



Height Reduction VIII

- THR can be easily performed when the expression consists of operations of the same type
- When expressions include operations of various types, THR can be executed by finding the best combination of the arithmetic properties of those operations and factorization techniques
- A THR algorithm can exploit commutative and associative properties of the operators in an expression tree by subtree swapping and rotation (left or right) with the goal of minimizing the tree height
- Applying THR may, in some cases, worsen the hardware implementation results, when some operations in the expression tree share hardware resources as their execution must be serialized

Height Reduction IX

- Suppose that we want to implement following expression in hardware

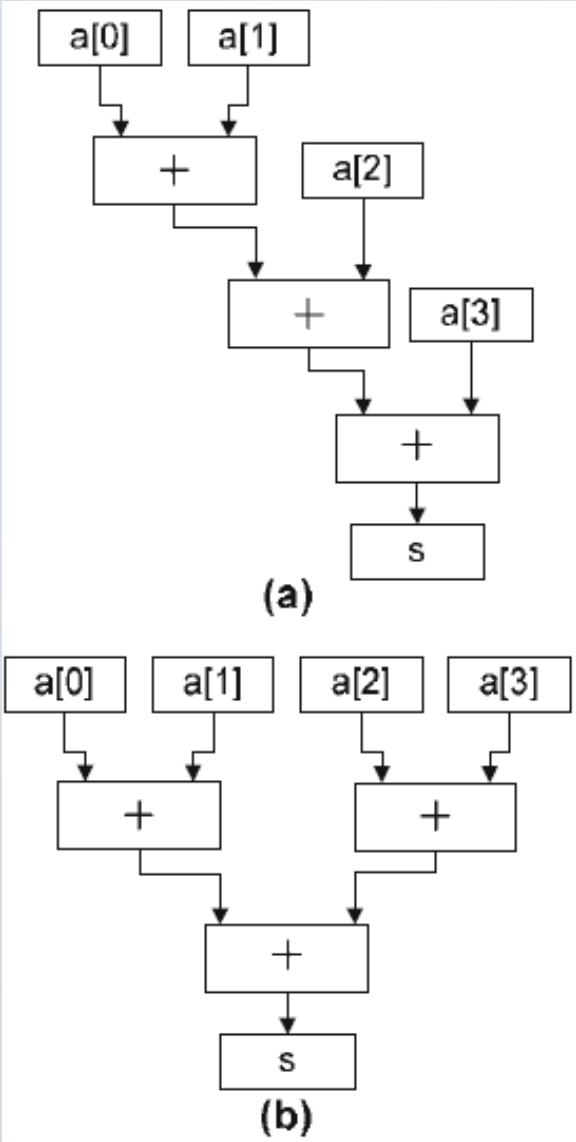
  s = a[0] + a[1] + a[2] + a[3]

- Schedule of naive implementation, without THR optimization is shown in Figure *a)*

- After performing THR, we get an implementation shown in Figure *b)*

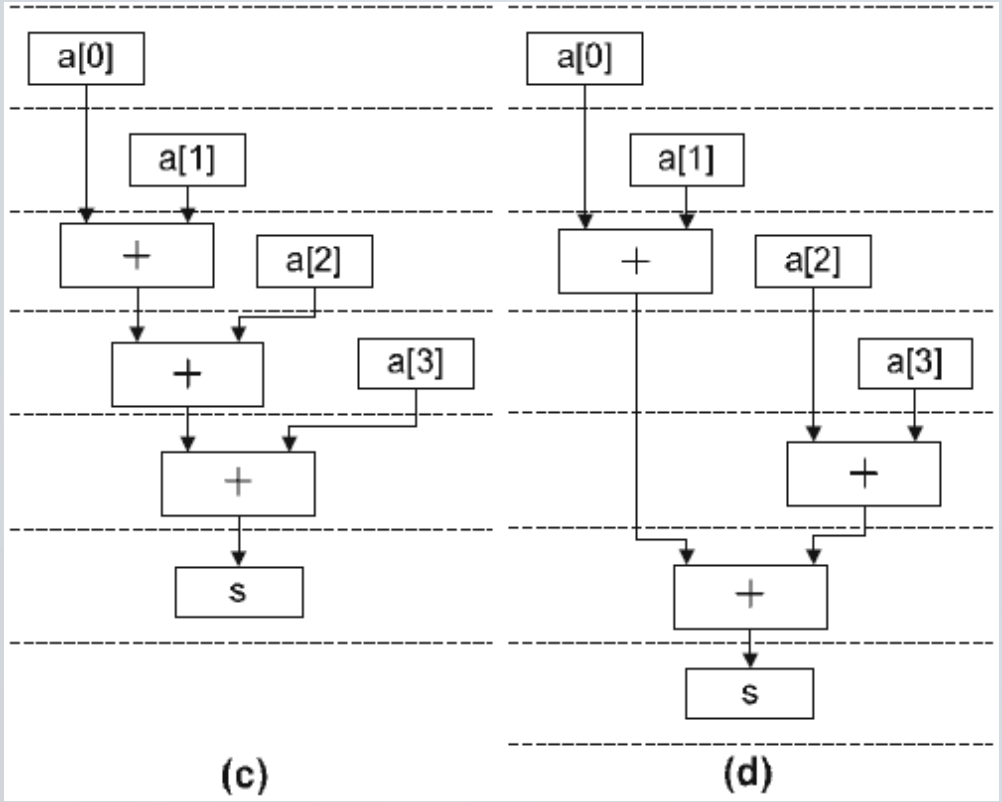
- Latency of schedule in Figure *a)* is Lat(mem_read) + 3*Lat(+), while latency of schedule in Figure *b)* is Lat(mem_read) + 2*Lat(+)

- However, schedule in Figure b) requires 4 simultaneous reads from the memory, to fetch the values *a[0]*, *a[1]*, *a[2]* and *a[3]*



Height Reduction X

- Suppose now that the array *a* is stored in one-port memory
- In this case schedule of naive implementation is shown in Figure *c)*, and schedule after performing THR is shown in Figure *d)*
- Latency of schedule in Figure *c)* is now 2*Lat(mem_read) + 3*Lat(+), while latency of schedule in Figure *d)* is 3*Lat(mem_read) + 3*Lat(+)!
- This example illustrates that THR must be employed carefully, taking in account overall system constraints



Code Motion I

- Code motion is a technique that changes the order of the execution of instructions in an algorithm by moving them either against the flow of control (hoisting) or along the flow of control (sinking)
- Code motion is used extensively across control-flow branches
- By moving an instruction in a shared section of the control-flow graph into disjoint sections of the control-flow graph, the compiler is trading off resources for concurrency
- The two instances of the instruction are potentially realized by distinct hardware operators and the overall execution path is shortened

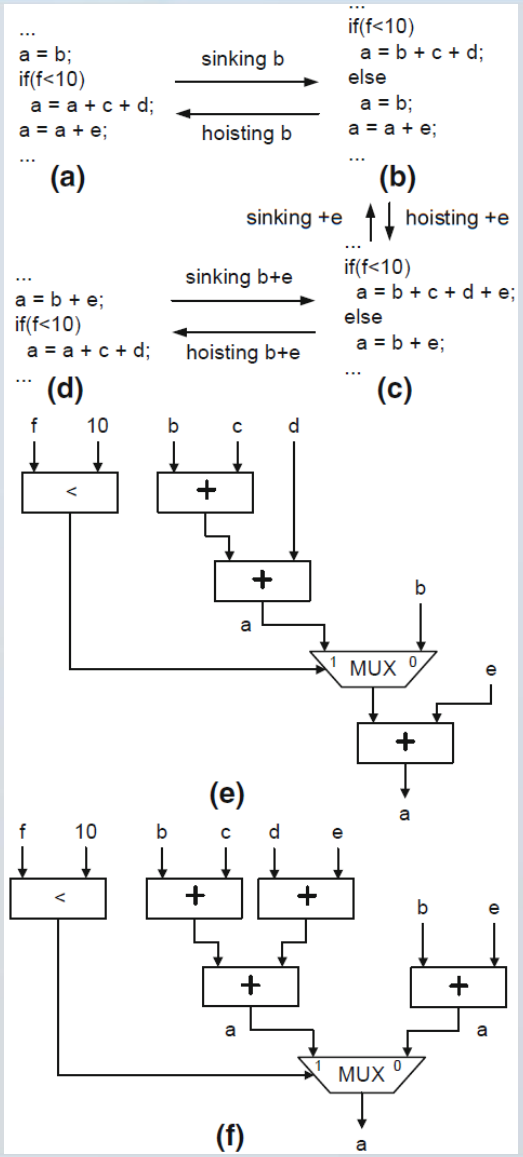
Code Motion II

- Figure on the right shows some examples of code hoisting and sinking (Figures *a-d*) and an example of the application of code hoisting and THR to reduce the critical path of the computation (Figures *e, f*)

- Hoisting +*e* transformation results in one more adder unit, but decreases the critical path length by the delay/latency of one adder unit, as shown in Figure *f)*

- Using code sinking, the code in Figure *c)* would be transformed into the code depicted in Figure *b)*, in this case increasing latency but saving hardware resources, as shown in Figure *e)*

- Other forms of code hoisting and code sinking can be applied to Figure *c)* to move upwards or downwards the subexpression b+e, respectively



Code Motion III

- In addition to its application within basic blocks or across control-flow constructs, code motion is also used in wider scopes
- In the context of loops, loop invariant code motion hoists to the loop preheader instructions that are always executed when the loop executes and that evaluate to the same values on every iteration of the loop
- This loop invariant code motion transformation has the same benefits than in traditional architectures as it reduces the number of times an instruction is executed
- Loop invariant code motion leads, in general, to hardware implementations with fewer hardware resources for the execution of the computations of the loop body, as well as to a possible benefit in terms of the length of schedule of the execution of the loop body

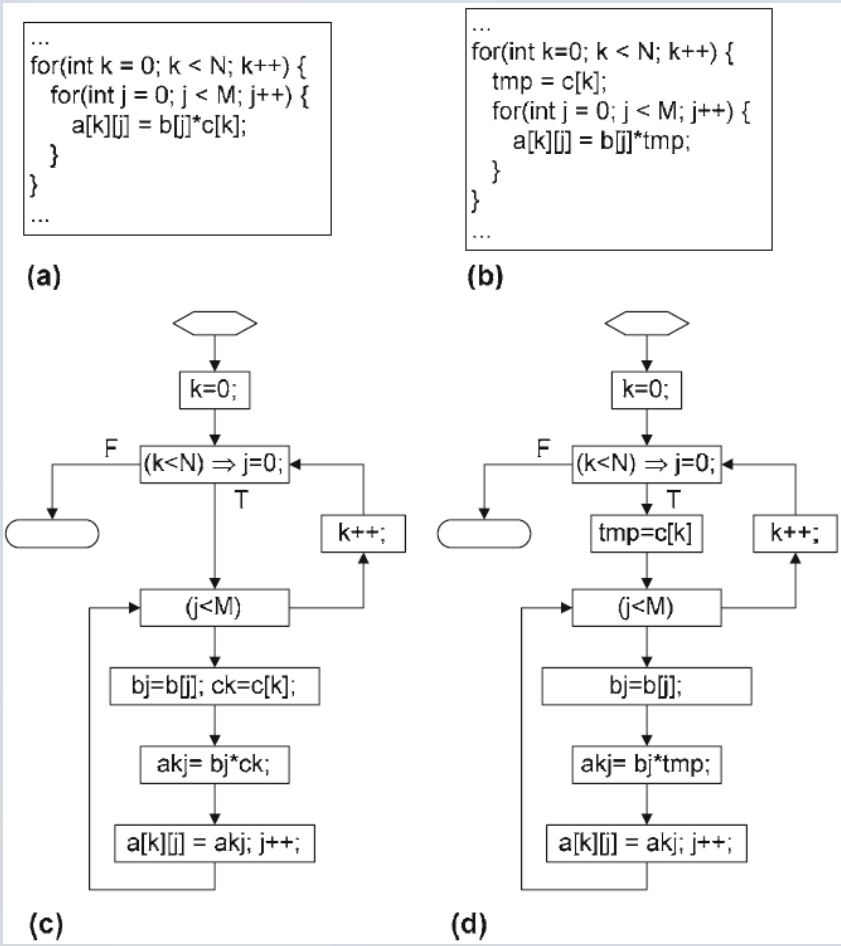
Code Motion IV

- In Figure *b)* the application of loop invariant code motion for the source code depicted in Figure *a)* is shown, where the loop-invariant statement *c[k]* is moved outside j loop

- In this example we assume that the target architecture has at least two independent memory modules to which the compiler can map distinct array variables b and c

- In a direct hardware implementation corresponding to the original algorithm, the execution would be able to concurrently access the arrays *b* and *c* as depicted in the corresponding ASM chart in Figure *c)*

- After the application of loop invariant code motion, the hardware implementation is forced to execute the memory accesses in disjoint steps as depicted in the ASM chart in Figure *d)*



Code Motion V

- Implementation shown in Figure *d)* has increased execution latency than the original implementation, shown in Figure *c)*
- This is because the execution latency of the *j* loop increases
- However, there is a reduction of the number of memory accesses, as *c[k]* is only performed once for each iteration of *k* loop
- This means that implementation from Figure *d)* is more power efficient than the implementation from Figure *c)*, since external memory accesses consume the most power, compared with other operations present in the original algorithm
- Furthermore, in the implementation from Figure *d)* memory accesses to *b[j]* and *c[k]* are now performed in disjoint steps, meaning that simpler, cheaper one-port memory could be used



Loop-Level Transformations

Loop-Level Transformations I

- Loop-Level transformations are used to transform loops that are present in the algorithm in order to match the instruction-level parallelism (ILP) in a given computation to the available resources in the target hardware architecture

- Loop-level transformations expose the available concurrency within the loops, e.g., by unrolling, and/or increase the required data availability, e.g., by tiling

- Loop-level transformations also enable and/or expose many opportunities for data management as described in the next section

- Tables on following two slides illustrate representative loop transformations, whose application can lead to more efficient hardware implementations

Representative Loop-Level Transformations I

Loop Transformation	Original source code	Transformed source code
Normalization	**for** (i=2; i<N; i++) sum += A[i-2];	**for** (i=0; i<N-2; i++) sum += A[i];

Unrolling (fully or by a factor *k*)	**for** (i=0; i<N; i++)
	  sum += A[i];
		for (i=0; i<N; i+=2) {
		  sum += A[i];
		  sum += A[i+1];
		}

Unswitching	**for** (i=0; i<N; i++) {
	  sum += A[i];
	  if (b) A[i] = 0;
	}
		if (b)
		  for (i=0; i<N; i++) {
		  sum += A[i];
		  A[i] = 0;
		}
		else
		  for (i=0; i<N; i++)
		  sum += A[i];

Reversal	**for**(i=1; i<N; i++) sum += A[i];	**for** (i=N-1; i>=0; i--) sum += A[i];

Representative Loop-Level Transformations II

Loop Transformation	Original source code	Transformed source code
Interchange/ reordering	**for** (j=0; i<M; j++) **for** (i=0; i<N; i++) sum += A[j][i];	**for** (i=0; i<N; i++) **for** (j=0; i<M; j++) sum += A[j][i];
Strip-mining (single nested loops)	**for** (i=0; i<N; i++) sum += A[i];	**for** (is=0; is<N; is+=S) **for** (i=is; is<min(N,is+S); i++) sum += A[i];
Tiling/blocking (generic nested loops)	**for** (j=0; j<M; j++) **for** (i=0; i<N; i++) sum += A[j][i];	**for** (jc=0; jc<M; jc+=B) **for** (ic=0; ic<N; ic+=B) **for** (j=jc; j<min(M,jc+B); j++) **for** (i=ic; i<min(N,ic+B); i++) sum += A[i];

Fusion/merging	**for** (i=1; i<N; i++)
	  sum += A[i];
	for (i=1; i<N; i++)
	  prod += B[i]*B[i];
		for (i=0; i<N; i++) {
		  sum += A[i];
		  prod += B[i]*B[i];
		}

Representative Loop-Level Transformations III

Loop Transformation	Original source code	Transformed source code

Fission/distribution	**for** (i=0; i<N; i++) {
	  sum += A[i];
	  prod += B[i]*B[i];
	}
		for (i=1; i<N; i++)
		  sum += A[i];
		for (i=1; i<N; i++)
		  prod += B[i]*B[i];

Splitting	**for** (i=0; i<N; i++) sum += A[i];	**for** (i=0; i<N/2; i++) sum += A[i]; **for** (i=N/2; i<N; i++) sum += A[i];

Peeling	**for** (i=0; i<N; i++)
	  sum += A[i];
		sum = A[0];
		for (i=1; i<N; i++)
		  sum += A[i];

Alignment	**for** (i=1; i<N; i++) {
	  b[i] = a[i];
	  d[i] = b[i-1];
	  c[i] = a[i+1];
	}
		d[1] = b[0];
		for (i=2; i<N; i++) {
		  b[i-1] = a[i-1];
		  d[i] = b[i-1];
		  c[i-1] = a[i-1];
		}
		b[N] = a[N];
		c[N] = a[N+1];

Loop-Level Transformations II

- A base loop transformation that simplifies the application of many other loop transformations is loop normalization

- In this loop iteration-space transformation, the compiler sets the loop control variable to exhibit an initial zero value and a unity increment step adjusting accordingly all the uses of the control variable in the loop body as well as the loop's upper bound expression

- We now illustrate the use of three common loop transformations that focus on increasing the available parallelism, namely:
  - Loop unrolling,
  - Loop tiling, and
  - Loop merging and distribution

Loop Unrolling I

- **Loop unrolling** is the most commonly used loop transformation when mapping loop computations to hardware
- The body of the unrolled loop, usually the innermost loop of a nest, is replicated, and the index expression corresponding to each of the unrolled iterations is propagated to the statements in each instance of the loop body as illustrated in Figure on the right
- By replicating the statements in the body of the loop, loop unrolling exposes more opportunities for the concurrent execution of the multiple instances of arithmetic operators corresponding to the various instructions in the statements of the loop, only subject to data or hardware resource dependences
- In addition to the increase in the potential for instruction-level parallelism, loop unrolling also decreases the iteration control overhead as the run-time tests used to determine if a given iteration of the loop is executed are partially or totally eliminated

```
…
#define W 3
…
int a[W][W], b[W][W], c[W];
…
for(x=0; x < W; x++) {
  sum = 0;
  for(y=0; y < W; y++) {
    sum += (a[x][y] * b[y][x]);
  }
  c[x] = sum;
}
…
```

(a)

```
…
#define W 3
…
int a[W][W], b[W][W], c[W];
…
for(x=0; x < W; x++) {
  sum = (a[x][0] * b[0][x]);
  sum += (a[x][1] * b[1][x]);
  sum += (a[x][2] * b[2][x]);
  c[x] = sum;
}
…
```

(b)

Loop Unrolling II

- The increase in the required resources to meet the demands of the operators in the unrolled loop may require a slight increase in storage to accommodate the many temporary register values used in the evaluation of the various operations

- Furthermore, the potential concurrent execution of many operators, and despite the potential for data reuse, increases the pressure on data bandwidth or data availability, measured on a per iteration basis

- As such, in some instances, loop unrolling might not be profitable, or the corresponding performance gains might not justify the additional required resources

Loop Unrolling III

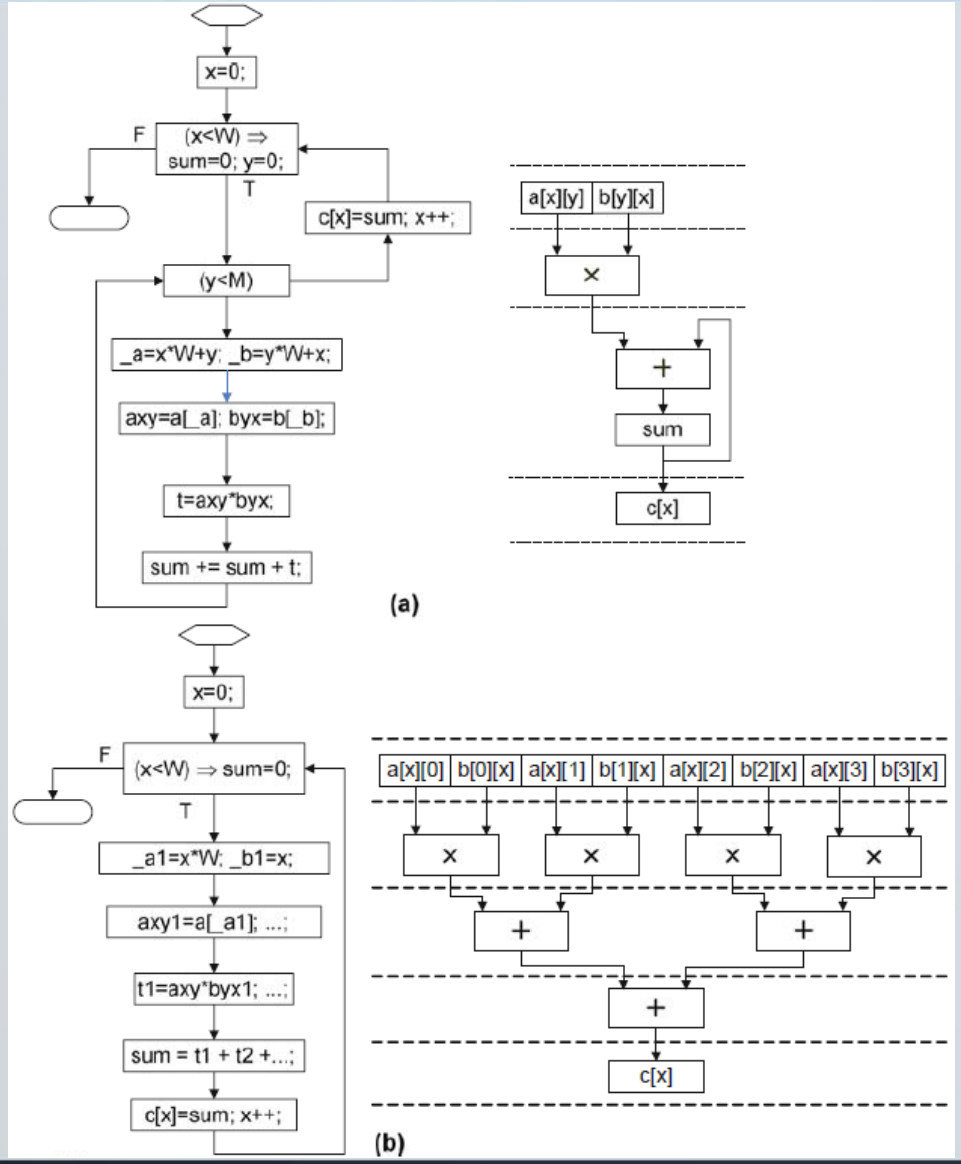
- A loop can be unrolled either partially or fully
- In partial loop unrolling, the body of the loop is replicated only *k* times with *k* less than the number of loop iterations to be executed.
- Common implementations of partial loop unrolling use an unrolling factor that evenly divides the number of loop iterations
- Otherwise, the transformed code will have to include an epilogue or additional control flow in the body of the loop
- When the loop bounds are not known statically, loop unrolling requires the use of control-flow constructs to determine if given iteration should be executed

Loop Unrolling III

- In fully loop unrolling the loop is completely unrolled, i.e., the body of the loop is replicated as many times as the number of iterations of the loop
- In this case, the loop bounds must be known statically, possibly by the use of constant propagation, as in the example depicted on Slide 48
- After loop unrolling, we can apply a wide range of other code transformations
- For example, exploiting commutative and associative properties to the example from Slide 48, we can rearrange the accumulations in the sum variable and apply THR while still exploiting the potential for concurrent execution of the three multiplications

Loop Unrolling IV

- Figure on the right depicts two illustrative hardware implementations
- Implementation based on the original algorithmic model is shown in Figure *a)*
- Implementation based on the transformed algorithmic model using loop unrolling and THR targeting an architecture with multiple hardware multipliers is shown in Figure *b)*

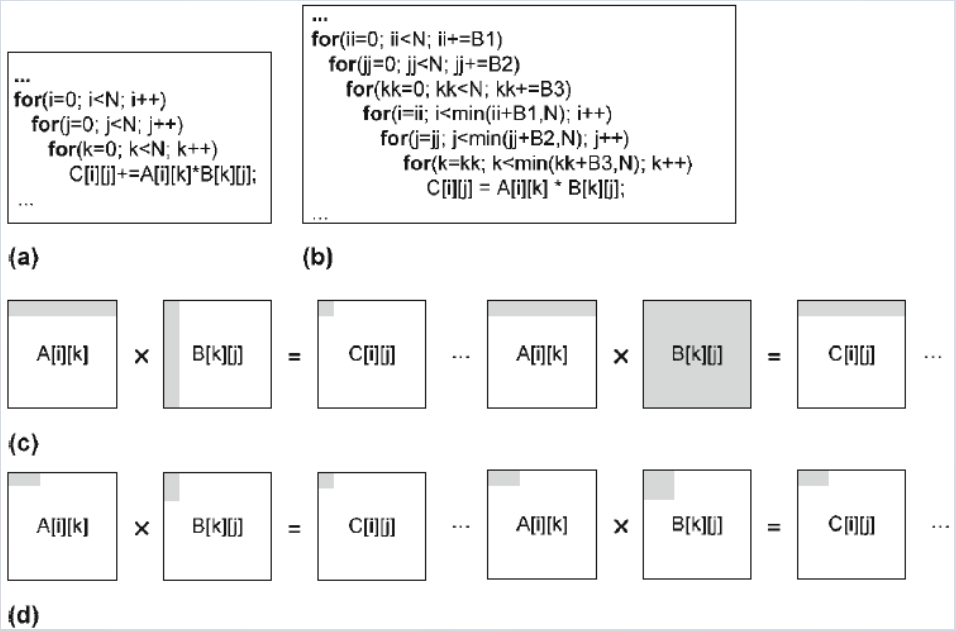


Loop Tiling and Strip-Mining I

- **Loop tiling**, or **blocking**, transforms the iteration space of the loop nest by structuring the execution of the loop into blocks/tiles of iterations of the original loop

- A tilled loop is thus structured as two loops, where an outer loop, called *control loop*, determines which of the blocks of iterations the inner loop executes

- Figure on the right depicts an example of loop tiling, where the iteration space of the original loop is split into blocks of size $B1 \times B2 \times B3$



Loop Tiling and Strip-Mining II

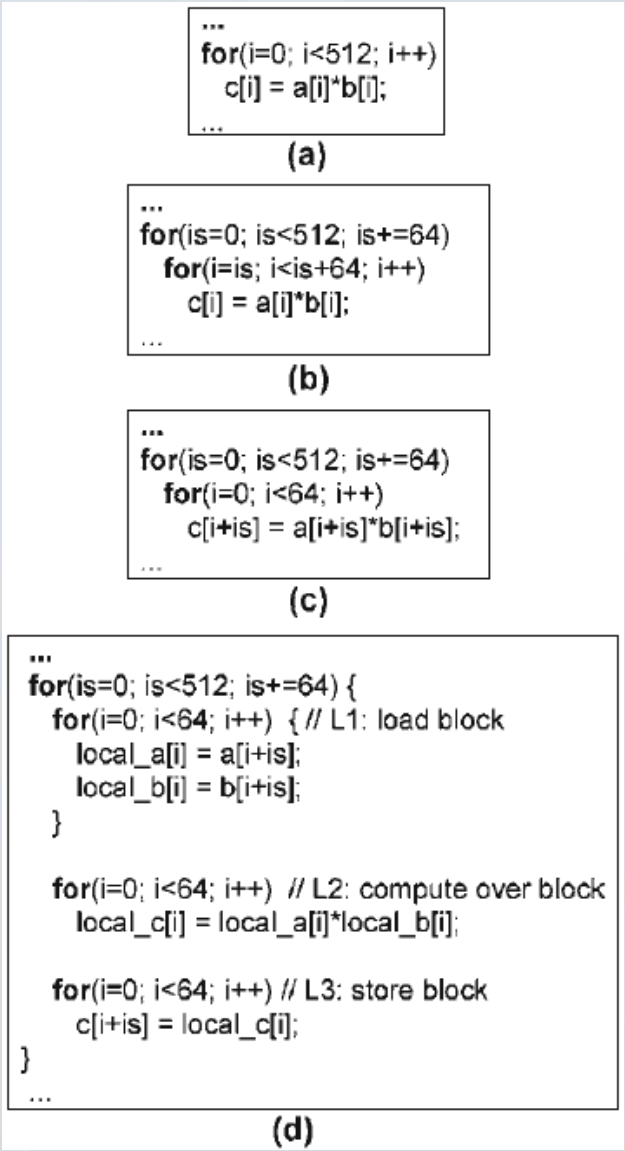
- As highlighted by previous example, loop tiling promotes locality, which is an important property to reduce cache hit misses when cache memories are used
- More importantly, in the absence of data dependences, invocations of the innermost loop in a tiled loop can be executed concurrently
- After loop tiling, we can distribute the computation associated with each iteration of the control loop (an invocation of the inner loop) across distinct FUs and distribute the corresponding data they manipulate between multiple distributed memories e.g., the block RAMs in FPGAs
- A particular case of loop tiling is **loop strip-mining**, where tiling is applied to singly nested loops, creating an inner loop responsible for the computation on each strip and an outer loop to control or traverse the various strips

Loop Tiling and Strip-Mining III

- The locality and coarse-grained concurrency make loop tiling or loop strip-mining particularly suited when targeting hardware implementation as highlighted by the example shown in Figure on the right

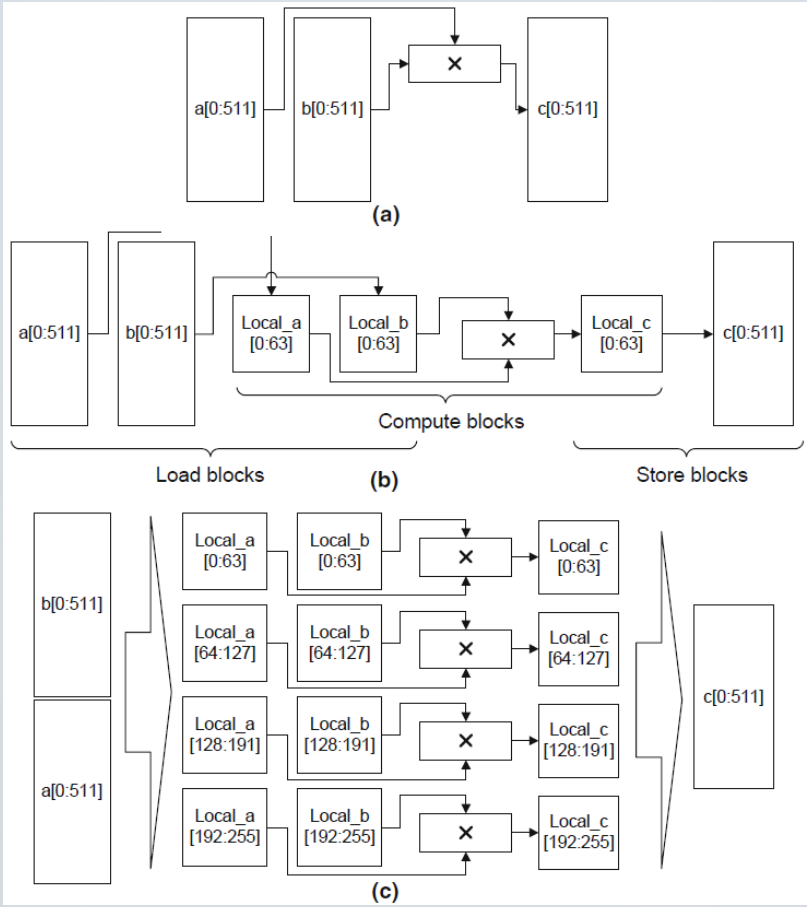
- Figure *b)* depicts the application of loop strip-mining to the source code in Figure *a)* whose normalization of the innermost loop is depicted in Figure *c)*

- The inner loop of the strip-mined loop can then be split into three consecutive loops using loop distribution (described in the next section) resulting in the code in Figure *d)*



Loop Tiling and Strip-Mining IV

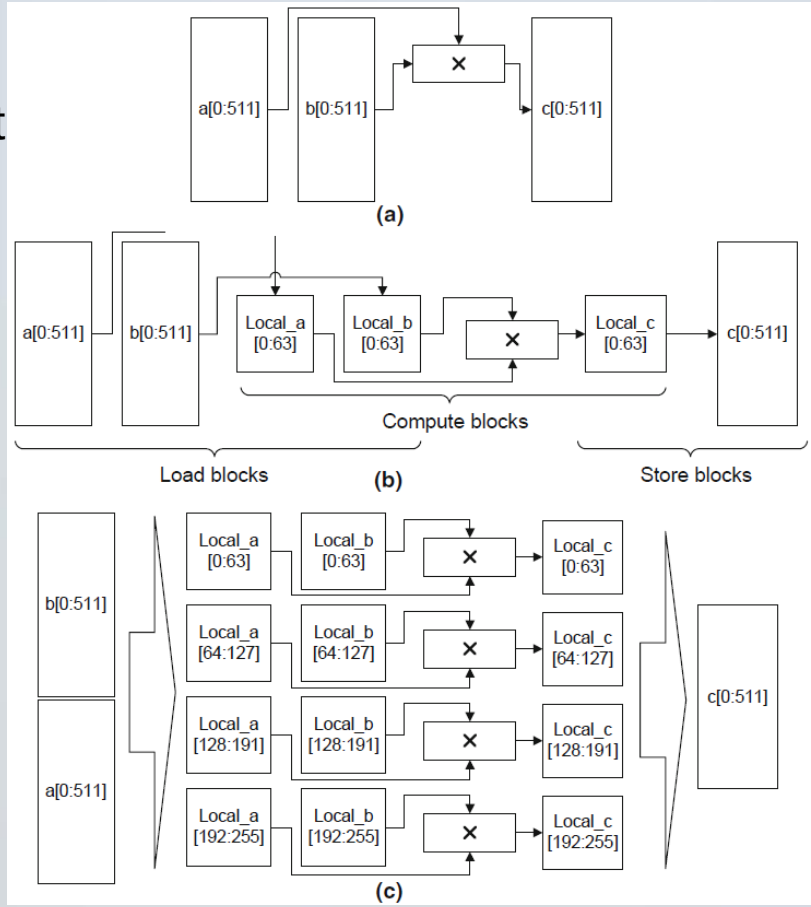
- In the first of these three loops, the computations access data in the two arrays *a* and *b* mapped to local storage
- In the second loop, the computations perform the multiplications using the data in internal storage and in the last loop the computation writes the results to the array *c* mapped to external storage
- A hardware implementation based on the transformed code in Figure *d)* from the previous slide is depicted in Figure *b)* to the right
- It can take advantage of the use of distributed local memories and of the splitting of the computations in three stages corresponding to the three loops: loading of a block of data, computing over a block, and storing a block of results
- The organization of the computation in stages suggests the application of coarse-grained pipelining execution, a technique described later



Loop Tiling and Strip-Mining V

- An alternative implementation strategy is to apply unrolling to the outer loops followed by strip-mining of the inner loops of the loop nest as depicted in Figure *c)*, resulting in a code that is easily vectorizable

- As it is also apparent in Figure *c)* the data in the input arrays *a* and *b* can be easily partitioned across two internal/external RAMs thereby increasing the availability of the data, and thus improving the overall performance of the implementation, even without resorting to pipelining execution techniques



Loop Tiling and Strip-Mining VI

- Overall, loop tiling/strip-mining transformations potentially increase data locality and enable the application of coarse-grained parallelization and pipelining execution techniques, across iterations of the control loops

- Loop tiling also exposes opportunities for array memory mapping, described later, as blocks of iterations of the tilled loops may access disjoint sets of data

- If these sets to are mapped to distinct physical memories, memory access contention is avoided

Loop Merging and Distribution I

- **Loop merging** or **loop fusion** merges two loops into a single loop
- The body of the fused loop consists of the concatenation of the loop bodies of the two original loops as depicted in the example in Figure on the right
- This transformation is trivially implemented when the loop bounds of the loops are identical and there are no data dependences between the two loops, as in the transformed code the sequences of the statements of the two original bodies execute in an interleaved order
- Loop fusion may increase the amount of available instruction level parallelism as the statements corresponding to the merged loop bodies can be concurrently executed, increasing also the possibility of resource sharing
- At the same time, loop fusion may increase the pressure on memory bandwidth and on register usage, thus requiring more registers/operations and memory accesses per iteration

```
for(i=0; i<N; i++)
    sum += a[i];
for(i=0; i<N; i++)
    prod += a[i]*a[i];
```

(a)

```
for(i=0; i<N; i++) {
    sum += a[i];
    prod += a[i]*a[i];
}
```

(b)

Loop Merging and Distribution II

- The dual loop transformation to loop fusion is **loop fission** or **loop distribution**
- Loop distribution splits a single loop into two loops with the exact same loop bounds but distributing the statements in the body of the original loop between the bodies of the two newly created loops
- Excluding output- (write-after-write, WAW) and anti-dependences (write-after-read, WAR), the transformation is always legal as the relative order in which the statements execute in the original loop is preserved in the transformed code
- As this transformation is the dual of loop fusion, we omit another illustrative example here



Data-Oriented Transformations

Data-Oriented Transformations

- We now describe three basic data-oriented transformations that are particularly suited when implementing algorithms in hardware, given the flexibility of organization and configuration of storage structures they allow, namely:
  - **Data Distribution**,
  - **Data Replication**,
  - **Data Packing/Unpacking**, and
  - **Data Reuse and Scalar Replacement**
- Many, if not all, of the data-oriented transformations exhibit a strong synergy with loop-level transformations, in particular for computations that manipulate array variables using affine index access functions

Data Distribution I

- **Data distribution**, commonly used for array variables, partitions each array into disjoint array subsets, each of which is then mapped to a distinct memory module
- This transformation, when combined with loop unrolling, improves array data availability and allows the generation of hardware implementations that can concurrently access the data without contention
- Data distribution does not increase the required storage needs as the original data is partitioned into disjoint data sets
- Other than a possible execution time overhead in reorganizing the data via distribution, data distribution increases the availability of data, provided the architecture has enough disjoint memory modules with adequate capacity to accommodate the partitions
- The use of distributed memories can be avoided when memory banks have enough memory access ports

Data Distribution II

- Figure on the right illustrates the application of loop unrolling, and data distribution for the *img* array variable
- The original *img* array is first partitioned into two distinct arrays, *imgOdd* and *imgEven*, which are then mapped to two different memories
- This memory mapping of the two arrays allows the two memory load operations, corresponding to the unrolled statement in the code shown in Figure *b)*, to be executed concurrently

```
…
type img[N][N];
…
for(j=0; j < N; j++) {
  …
  for(i=0; i < N; i++) {
    … = img[j][i];
  }
  …
}
```

(a)

```
…
type imgOdd[N][N/2], imgEven[N][N/2];
…
for(j=0; j < N; j++) {
  …
  for(i=0; i < N; i+=2) {
    … = imgOdd[j][i/2];
    … = imgEven[j][i/2];
  }
  …
}
```

(b)

Data Replication I

- Data replication creates various copies, or replicas, of specific data items which are then mapped to distinct storage structures
- This transformation thus increases the availability of the data by allowing concurrent data accesses at the expense of increased storage use
- Figure on the right illustrates a combined application of loop unrolling and data replication for the *img* array in the original example code
- The code is first unrolled by a factor of 2, generating two statements that access the *img* array
- These accesses are then matched with two distinct array copies *imgA* and *imgB* of the original *img* array

```
…
type img[N][N];
…
for(j=0; j < N; j++) {
  …
  for(i=0; i < N; i++) {
    … = img[j][i];
  }
  …
}
```

(a)

```
…
type imgA[N][N], imgB[N][N];
…
for(j=0; j < N; j++) {
  …
  for(i=0; i < N; i+=2) {
    … = imgA[j][i];
    … = imgB[j][i+1];
  }
  …
}
```

(b)

Data Replication II

- When applied to array variables, data replication has to be exercised with caution as it increases the availability of data at the expense of potentially substantial increase in allocated storage
- Its applicability may be therefore limited to small arrays that are immutable for a specific locus of computation
- For example, while an array variable is modified throughout the entire program, in a specific loop nest it may only be read
- This is a common case with algorithm parameters or computational coefficient loaded at the beginning of the program and never modified afterwards
- In the presence of mutable data, replication raises the issue of consistency
- The execution must ensure all replicas are updated with the correct values before the multiple copies can be accessed, and that any additional computation executed afterwards observes any possibly modified values

Data Packing/Unpacking I

- Packing and unpacking of data items allows the implementation to reduce the number of memory accesses
- When the basic memory transfer unit or block is larger than the individual data items the computation manipulates, the implementation can associate many data items to a single memory word
- An individual memory access thus fetches multiple data items, thereby reducing the number of memory accesses when the computation requires all the items need to be fetched
- This mapping technique is particularly beneficial for computations that access consecutive array data elements or when the stride is known at compile time, and the compiler customizes the data layout to match the data access patterns of the computation

Data Packing/Unpacking II

- Figure on the right illustrates the application of packing and unpacking for a computation that operates on 5-bit array elements
- In Figure *b)* the designer has packed every six consecutive elements of the *a* array into a single 32-bit element of the new array *a*
- It then laid out the new *a* array in memory by padding the two most significant bits in each element
- For each read access the compiler translates the basic memory access with mask and shift arithmetic and logic bit-level operations as depicted in Figure *b)*, revealing the huge saving as all six individual data items are now accessed by a single memory read operation

```
int:5 a[N]; // array of 5-bit integers

if((N % 6) == 0) {
  for(i=0; i < N; i += 6) {
     … = a[i+0];
     … = a[i+1];
     … = a[i+2];
     … = a[i+3];
     … = a[i+4];
     … = a[i+5];
  }
}
```

(a)

```
int:32 a[N]; // array of 32-bit integers

if((N % 6) == 0) {
  for(i=0; i < N; i += 6) {
    w  = a[i];
    … = (w & 0x0000001F):5;
    … = ((w & 0x000003E0) >> 5):5;
    … = ((w & 0x00007C00) >> 10):5;
    … = ((w & 0x000F1000) >> 15):5;
    … = ((w & 0x01F00000) >> 20):5;
    … = ((w & 0x3E000000) >> 25):5;
  }
}
```

(b)

Data Packing/Unpacking III

- Write operations, however, are more complicated as the implementation may have to first read the memory word, mask the bits corresponding to the specific data item and them write it back to memory

- Despite the memory savings and the reduction of the number of memory accesses, unpacking operations may impose non-negligible overheads in both latency and hardware resources.

- In most hardware architectures, however, the unpacking operations in the loop body of Figure *b)* can be performed very efficiently via interconnection resources (wires)

```
int:5 a[N]; // array of 5-bit integers

if((N % 6) == 0) {
  for(i=0; i < N; i += 6) {
    … = a[i+0];
    … = a[i+1];
    … = a[i+2];
    … = a[i+3];
    … = a[i+4];
    … = a[i+5];
  }
}
```

(a)

```
int:32 a[N]; // array of 32-bit integers

if((N % 6) == 0) {
  for(i=0; i < N; i += 6) {
    w  = a[i];
    … = (w & 0x0000001F):5;
    … = ((w & 0x000003E0) >> 5):5;
    … = ((w & 0x00007C00) >> 10):5;
    … = ((w & 0x000F1000) >> 15):5;
    … = ((w & 0x01F00000) >> 20):5;
    … = ((w & 0x3E000000) >> 25):5;
  }
}
```

(b)

Data Reuse and Scalar Replacement I

- In many computations, particularly in the context of digital image and signal processing, data values are often reused
- Examples of this reuse occur when the computation repeatedly uses coefficients of a signal transformation or reuses values when repeatedly accessing overlapped sections of an array
- A design can exploit this data reuse by selectively choosing which data values are reused in a given computation and saving or caching them in scalar variables, which are mapped to registers, in a transformation known as **register promotion**
- The designer transforms the code to save the reusable values in registers (or internal RAMs) the first time the computation accesses them
- The values are then reused for the remainder of the computation

Data Reuse and Scalar Replacement II

- This caching of data “locally” in registers or RAMs has the potential to improve the overall performance of the corresponding hardware implementation by two main factors
- First, the use of registers substantially decreases the data access latency
- In hardware architectures where registers and RAMs are distributed throughout the architecture, reusing data in discrete registers tremendously increases the amount of data bandwidth as all the registers can be accessed concurrently
- Second, by reusing data internally, the implementation can drastically reduce the number of external memory accesses

Data Reuse and Scalar Replacement III

- Figure on the right illustrates the application of scalar replacement to the *vec* and *coeff* array variables
- In the original code, in Figure *a)*, the *N* locations of the *vec* array are repeatedly accessed on every iteration of the *j* loop
- A way to capture this reuse is to peel the first iteration of the *j* loop (for *j*=0) and during the *i* loop corresponding to this first iteration, save the *N* elements of each row *k* of *vec* in the auxiliary vector *vec_sr*
- The values in *vec_sr* are mapped to a set of *N* registers or an internal RAM module in the target architecture and reused through the remainder *N-1* iterations of the *j* loop
- Similarly, the location *coeff[k]* is repeatedly accessed throughout the entire computation and can be saved (outside the *j* loop) into a scalar variable

```
type coeff[M];
type vec[N];
…
for(k=0; k < M; k++) {
  …
  for(j=0; j < N; j++) {
    …
    for(i=0; i < N; i++) {
      … = vec[k][i] * coeff[k];
    }
    …
  }
}
```

(a)

```
type coeff[M], coeff_sr;
type vec[M][N], vec_sr[N];
…
for(k=0; k < M; k++) {
  …
  /* i,j loop invariant */
  coeff_sr = coeff[k];
  /* iteration for j = 0 */
  for(i=0; i < N; i++) {
    vec_sr[i] = vec[k][i]; /* saving data */
    … = vec_sr[i] * coeff_sr;
  }
  …
  for(j=1; j < N; j++) {
    …
    for(i=0; i < N; i++) {
      … = vec_sr[i] * coeff_sr;
    }
    …
  }
}
```

(b)